

软件分析论文中的“坏味道”

蒂姆·孟席斯

美国北卡罗来纳州立大学计算
机科学系

马丁·谢泼德

布鲁内尔软件工程实验室 (BSEL)
伦敦布鲁内尔大学计算机
科学系
UB8 3PH, 英国

抽象的

背景：数据分析在支持循证软件工程方面的应用正在快速增长。然而，复杂的技术、多样化的报告标准以及对潜在现象的理解不足，引发了人们对研究可靠性的担忧。

目标：我们的目标是软件分析研究（计算实验和相关性研究）的生产者和消费者提供指导。方法：我们建议使用“坏味道”，即敏捷软件社区中流行的深层问题的表面迹象，并思考它们如何在软件分析研究中体现出来。

结果：我们在软件分析论文中列出了 12 种“坏味道”（并通过示例展示了它们的影响）。

结论：我们认为“坏味道”的比喻很有用。因此，我们鼓励就软件分析研究的有效性因素展开更多讨论（因此我们预计我们的清单会随着时间的推移而不断完善）。

1. 简介

自从 Kitchenham 等人发表了开创性的文章以来，将软件工程确立为一门基于证据的学科的动力日益增强。[\[56\]](#)与此同时，公开的软件数据和复杂的数据分析方法的数量也大幅增长。这导致基于实证、数据驱动的研究（通常称为软件分析）的数量和范围急剧增加。

通常，软件分析研究试图将大量低价值数据提炼成小块的高价值信息。研究更多地以数据为导向，而非理论驱动。例如，在研究了许多软件项目之后，某些编码风格可能会更容易出现错误。通过操纵这些编码风格进行后续实验，我们可能会相信其中存在因果关系。因此，我们可能会

建议避免某些编码风格（需遵守各种与上下文相关的注意事项）。然而，缺乏理论可能会带来挑战。特别是，缺乏强有力的先验信念可能会使在数百万种可能性中进行选择或评估变得困难。[24，二十八]。

到目前为止，一切都很好。不幸的是，软件工程（SE）内部对许多此类结果的可靠性表示担忧[93，53] 以及更广泛的其他实验和数据驱动学科，如生物医学科学[50，31] 这已经达到了实验心理学研究人员所说的“复制危机”的程度。这反过来又引发了研究有效性的质疑。

可以说，软件工程中的情况并无不同。在一项关于软件工程实验（1993-2002）的重要系统综述中，Kampenes 等人[54] 发现，心理学领域也报告了类似的效应大小。2012 年，我们在该期刊的特刊上发表了一篇社论。实证软件工程软件工程预测中可重复结果的研究[77] 主要关注的是“结论的不稳定性”。在那篇社论中，我们提出了对许多情况的担忧，即研究 1 得出 X 结论，而研究 2 得出 -X、这些看似矛盾的现象无法解决，让我们陷入了僵局。显然，我们需要更好的方法来开展和报告我们的工作，以及如何审查他人的工作。

在本文中，我们将探讨可以从其他学科中学习到什么，以及应该应用哪些具体经验来改进软件分析类研究的报告。“坏味道”是一个来自敏捷社区的术语。根据 Fowler [5]，坏味道（又称代码味道）是“通常与更深层次的问题相对应的表面迹象”。参见表格1了解本文中发现的不良气味的摘要。

正如福勒所说，我们认为本文列出的“异味”并不一定意味着必须否定基础研究的结论。然而，我们认为这些“异味”提出了三个潜在的担忧。

1. 对于作者来说，它们降低了出版的可能性。
2. 对于读者来说，它们对结果的可靠性（即其一致性）和有效性（即分析的正确性或它如何很好地捕捉我们认为它所代表的那些结构）提出了警告。
3. 对于科学而言，它们阻碍了通过元分析汇集结果和构建知识体系的机会。

因此，本文的目标是识别“气味”，以便（i）鼓励更广泛的社区辩论和（ii）协助识别潜在问题和相关补救措施。

本文的其余部分安排如下。本节的其余部分讨论了范围，然后解答了有关这项工作的两个常见问题。2 探讨撰写“优秀”文章的历史研究和方法论指导，以及其他数据驱动学科如何解决实验可靠性问题。接下来，在3我们描述了识别“坏味道”的方法。接下来是4列出了一些关键症状或“气味”的大致情况

表 1：软件分析不良气味摘要

	“坏的 闻”	核心问题	影响	补救
1	不跨 测试	软件分析相当于“针头上能跳出多少个天使”。这可能是由于理论的缺失造成的。	对软件工程影响微乎其微的研究	与从业人员对话[6, 70, 51]
2	不使用 有关的工作	不了解 (i) 研究问题和/或 (ii) 相关技术的最新进展的相关工作。	(i) 无意的复制 (ii) Not using state of the art methods (iii) Using unchallenging / outdated benchmarks	Read! Utilize systematic reviews whenever possible. Search for relevant theory or theory fragments. Utilize technical guidance on data analytics methods and statistical analysis. Speak to experts.
3	Using deprecated or suspect data e.g., <i>D</i> has been widely used in the past ...	Using data for convenience rather than for relevance	Data driven analysis is undermined [2]	Justify choice of data sets wrt the research question.
4	Inadequate reporting	Partial reporting of results e.g., only giving means without measures of dispersion and/or incomplete description of algorithms	(i) Study is not reproducible [72] (ii) metaanalysis is difficult or impossible.	Provide scripts / code as well as data. Provide raw results [41, 79].
5	Under-powered studies	Small effect sizes and little or no underlying theory	Under-powered studies lead to over-estimation of effect sizes and replication problems	Analyse power in advance. Be wary of searching for small effect sizes, avoid Harking and p-hacking [15, 53, 79].
6	$p < 0.05$ and all that!	Over-reliance on, or abuse of, null hypothesis significance testing	A focus on statistical significance rather than effect sizes leading to vulnerability to questionable research practices and selective reporting [53]	Report effect sizes and confidence limits [33, 65].
7	Assumptions of normality and equal variances in statistical analysis	Impact of heteroscedasticity and outliers e.g., heavy tails ignored. Assumptions of analysis ignored.	Under-powered studies lead to over-estimation of effect sizes and replication problems.	Use robust statistics / involve statistical experts [58, 110].
8	No data visualisation	Simple summary statistics e.g., means and medians may mask underlying problems or unusual patterns.	Data are misinterpreted and anomalies missed.	Employ simple visualisations for the benefit of the analyst and the reader [80, 44].
9	Not exploring stability. Only reporting best or averaged results.	No sensitivity analysis. p-hacking	The impact of small measurement errors and small changes to the input data are unknown but potentially substantial. This is particularly acute for complex models and analyses.	Undertake sensitivity analysis or bootstraps/randomisation to understand variability. Minimally report all results and variances [73, 87].
10	Not tuning	Biased comparisons e.g., some models / learners tuned and others not. Non-reporting of the parameter settings, the tuning effort / expertise required.	Under-estimation of the performance of un-tuned predictors. Inability to reproduce results.	Read and use technical guidance on data analytics methods and statistical analysis [99, 37]. Unless relevant to the research question avoid off-the shelf defaults for sophisticated learners.
11	Not exploring simplicity	Excessively complex models, particularly with respect to sample size	Dangers of over-fitting	Independent validation sets or uncontaminated cross-validation [18].
12	Not justifying choice of learner	Permuting / 重组 learners to make ‘new’ learners in a quest for spurious novelty	“无趣结果”的报告不足或不报告导致对效应大小的估计过高。	原则性地生成研究问题和分析技术。完整的报告。

重要性顺序。最后，在第5我们考虑对未来的影响，以及我们研究界如何共同承担这些“气味”，重新确定优先顺序、重新排序、添加和减少“气味”。

1.1. 范围

软件分析研究相对较新且发展迅速，因此我们的许多建议都与讨论从软件项目数据中进行归纳分析的文章有关。我们通过实验、准实验和相关性研究来讨论实证数据分析的问题[91]，而不是与定性研究相关的问题。有关涉及人类参与者的定性研究和实验的技巧和陷阱，请参阅其他文章，例如，[102，二十五，60，84]。

最后，为了界定本文的范围，我们认为有必要补充一些关于理论在软件分析类研究中的作用的评论。当然，许多分析研究本质上是数据驱动或归纳的。

在这种情况下
立场理论或演绎推理的作用将是次要的。这当然
这引出了一个问题：什么是理论。遗憾的是，对于理论的构成，人们的共识有限。[97，52]，除了它可能包含结构、结构之间的关系（可能是因果关系）、范围和可能的命题[101]大多数评论家认为，理论应用率较低，可以进一步提升。为了实现这一目标，Stol 和 Fitzgerald [101] 探索可能以命题形式出现的理论碎片的概念。那些可检验的碎片可能形成假设，即从理论到经验世界的机制，以及作为从经验观察回归理论的手段的推定命题。

话虽如此，我们并没有对软件工程理论进行详细的论述。数据分析研究几乎总是轻理论的，因为它们的方法采用归纳法。是否使用理论，将高度依赖于具体情境。目前，对于理论（或相应的模型）应该是什么样子，尚无共识。最后，正如文中所述，目前已经存在一些很好的处理方法（参见 [97，52，101]）。

1.2. 但这篇文章有必要吗？

简而言之：这篇文章有必要吗？这里所有的东西不都是“众所周知”的材料吗？

In response, we first note that, over some decades, the authors have been members of many conference and journal review panels. Based on that experience, we assert that demonstrably everybody does not know (or at least apply) this material. Each year, numerous research articles are rejected for violating the recommendations of this article, or worse still, some slip through the net. In our view many of those rejections are avoidable. We believe, many of the issues raised by this article can be repaired before submission – some with very little effort.

These difficulties are compounded by the fact that good practice evolves. For example, whilst the statistical education of many engaged in empirical software engineering research is grounded in classical statistics there have been more recent developments such as robust approaches [57], use of randomisation and bootstrap [16] and Bayesian techniques [39]. Likewise, there are significant and recent developments in machine

learning which, sometimes, authors overlook. For example, we have had on occasions to review and reject papers that are some very small delta to an article one of us wrote in 2007 [76]. Since that paper was written, much has changed¹ so a mere increment on that 2007 paper is unlikely to be a useful contribution or an acceptable publication.

We also observe that, on occasions, we find some reviewers who have the role of gatekeepers to scientific publication, also seem unaware of some, or much of this material. Thus there exists some research that should not be published, at least in its present form, that is published [58, 53]. This harms the progress of science and our ability to construct useful bodies of knowledge relevant to software engineering.

A particularly marginalized group are industrial practitioners trying to get their voice heard in the research literature. Unless that group knows the established norms of publication, their articles run the risk of rejection. We note that those norms of publication are rarely recorded – a problem that this article is attempting to solve. Hence, for all the above reasons, we argue that this article is necessary and useful.

1.3. Should papers be rejected if they have bad smells? It is

important to stress that this paper:

- is primarily intended as *guidance* for authors;
- is not intended to be used by reviewers as a mindless checklist of potential reasons-to-reject a paper. Our bad smells are *indicative* rather than *definitive* proof of a potential issue. Hence we stress that *their presence alone is not a reason for reviewers to reject a paper*. Context is extremely important.

本文的一位审稿人非常友好地就这一点进行了阐述。他评论道（我们同意），审稿人往往会变得过于严格——甚至会因为其中一点不符合某些指导原则而拒绝那些包含非常有趣和重要观点的论文。这种情况在很少有修订周期的会议中尤为严重。审稿人不应盲目地强制采纳本文提出的所有观点，还应敏感的与研究背景相关；某些“坏味道”可能不适用（或相关性较低）。此外，我们希望避免采用这样一种观点，即对完美的要求会阻碍有用、开创性或相关（但略有不完美）的研究的发表。

2.相关工作

软件工程 (SE) 研究人员时常会提出这样一个问题：一篇好文章应该具备哪些特质？例如 Shaw 的文章 [92] 早在 2003 年就已提出，最近 Theisen 等人又对其进行了重新审视。[107] Shaw 表示，SE 研究人员面临的一个难题是

¹例如，改进的评估技术和实验设计；SE 和机器学习的新问题领域（例如[88]）；新颖且富有洞察力的新评估措施（参见[49]或[中讨论的多目标方法81]；并挑战那些速度太慢以至于无法重现先前结果的新学习者（参见[三十五]）。

以及作者面临的一个问题是，目前尚无完善或公认的研究开展和呈现指南。尽管近年来人们越来越重视证据，但我们认为情况并未发生太大变化。她还指出，许多文章在验证方面存在不足，而Theisen等人的报告则指出，审稿人的期望值有所提高。鉴于软件分析文章的实证基础，它们在这方面可能更具优势，然而，统计分析并非总能很好地进行，正如我们的文章将要展示的那样。

对实证研究的统计质量也进行了各种审计[58]并发布了关于分析的指南和建议[58, 111]这些指南受到欢迎，因为总体而言，软件工程师接受的统计学家正规培训有限。需要注意的是，当研究人员发表方法论指南时，这些文章的引用次数往往很高。²，例如：

- “软件工程实证研究初步指南”，*IEEE 软件工程学报*[58]（自2002年以来已被引用1430次）。
- “软件缺陷预测的基准分类模型：提出的框架和新发现”*IEEE TSE*2008年[67]（自2008年以来已被引用765次）。
- “《软件工程案例研究的开展和报告指南》，*实证软件工程杂志* [86]（自2009年以来已被引用2500次）。
- “ICSE'11上的“使用统计测试评估软件工程中的随机算法的实用指南”[4]（自2011年以来已被引用500次）。

尤其令人担忧的是，我们能否重复实验结果（或可靠性），许多人认为这是科学的基石。Nosek等人在实验心理学领域对此进行了系统的研究。[83]一项涵盖100个实验的大型重复研究，发表在顶尖心理学期刊上，该研究得出结论：“重复效应只有原始效应的一半”，并且尽管97%的原始研究取得了统计学显著结果，但只有36%的重复实验达到了统计学显著水平。研究人员开始讨论“重复危机”。然而，值得注意的是，关于究竟什么构成了对原始研究的验证，却鲜有讨论。[100]而对于效力不足的研究（这似乎是很典型的情况），结果很可能受抽样误差的影响。尽管如此，似乎仍有理由感到担忧，这也引发了最近一些指导性文章，例如Munafò等人。[79]。

尽管这一“复制危机”在报道时引发了人们深刻的反思，但进展却不如我们所希望的那样，例如，在认知神经科学和实验心理学领域，研究能力仍然很低，并且在50年内基本没有变化。[104]。Smaldino和McElreath [98]认为，鉴于研究人员的激励机制——特别是出版物是职业发展的主要因素——低质量研究的出现并不奇怪。

²引用计数摘自2018年11月的Google Scholar。

仍在进行、提交和发表。正如他们所说，尽管这并非“故意作弊或偷懒”，但也解释了为什么科学实践的进步仍然缓慢。

我们相信，虽然激励机制显然至关重要，并会影响科研行为，但我们发现的许多问题只需付出相对较少的努力即可得到解决。我们也希望，“做好科研”的内在动力能够满足我们科研界的绝大多数人的需求。

3. 我们的方法

我们的目标是帮助研究人员开展更好的研究并撰写更好的论文。为了实现这一目标，我们提出了一个由两部分组成的提案：

1. 本提案的一部分内容是在主要的软件工程会议上举办研讨会，探讨和记录分析和写作方面的最佳和最差实践。因此，本文旨在招募同行参与此目标，从而增加思考和评论这些问题的研究人员的数量。
2. 该计划的另一部分是本文本身。我们希望这些想法本身能够发挥作用，同时也希望 (a) 本文能够激发关于最佳和最差论文写作实践的更广泛的讨论；(b) 后续文章能够扩展和改进我们以下提供的指导。

为了使本文更有效性，我们采取务实的方法。我们列出了软件分析论文中存在的问题，希望这些问题能够有所帮助，但并非详尽无遗。本文的详细程度旨在说明，而非深入探讨。为了简洁起见，我们将重点限制在数据驱动的研究上，因此，例如，案例研究和涉及人类参与者的实验等内容将被排除在外。

有些“异味”更加不言而喻，因此需要相对较少的解释。相比之下，其他一些异味则更加微妙，因此我们会投入额外的空间来说明它们。我们会使用示例来帮助说明这些异味及其与实际问题的关系。我们并不打算将示例作为存在性证明，并且无论如何，将个别作者单挑出来作为普遍错误的例子都是不公平的。

这份“气味”清单是两位作者通过讨论和辩论得出的。然后，我们交叉核对了这份清单（见表2）运用实证研究质量工具，[二十九] 用于评估文章质量是否适合纳入软件工程系统评价 [55，第 7 章]。因此，我们看到我们的清单涵盖了质量工具确定的所有四个质量领域，并且对其覆盖范围更有信心。

4. 揭秘“恶臭”

本节讨论表中列出的不良气味¹在开始之前，我们注意到，虽然以下一些“气味”特定于软件分析，但其他一些气味则普遍存在于许多科学研究中。我们将它们全部纳入，因为它们都是关于良好科学和有效科学交流的重要问题。

表 2：交叉引用研究领域与恶臭

研究领域[55]	质量仪器问题[二十九, 表3]	臭气
报告-质量	这篇文章是基于研究的吗? 研究目标是否清晰? 研究背景是否充分描述? 研究结果是否清晰表述?	是的, 定义为 我们的范围 1 4 4、11
严格研究设计的 细节	研究设计是否合理? 招募策略是否恰当? 治疗组与对照组相比如何? 收集的数据是否解决了研究问题? 数据分析是否足够严谨?	5 没有人类同行 参与者, 但是 更一般地: 1、3 2、10 3 6、7、8、9、10
可信度-该研究的结果 是否有效且有意义?	研究人员和参与者之间的关系是否得到充分考虑?	没有人 参与者
关联-学习实践	该研究有研究价值还是实践价值?	1、11

4.1. 不有趣

所有科学研究都应力求有效可靠。但重要的是, 研究也应有趣(对更广泛的受众而言, 而不仅仅是作者)。这并不是在号召大家进行古怪或不切实际的研究, 但请记住, 我们的学科是软件 工程. Potentially somebody must be invested in the answers, or potential answers. One rule-of-thumb for increasing the probability that an article will be selected is validity plus *engagement* with some potential audience.

So what do we mean by “interesting”? The article needs a motivation or a message that beyond mundane. More specifically, *articles should have a point*. Research articles should contribute to a current debate. This could be via theory or with respect to the results of other studies. If no such debate exists, research articles should motivate one. For example, one could inspect the current literature and report a gap in the research that requires investigation. But to be clear, this does not preclude incremental research or even checking the reproducibility of results from other researchers, particularly if those results may have strong practical significance.

Note that problem of *not inspiring* is particularly acute in this era of massive availability of data and new machine learning algorithms; not all possible research directions are actually interesting. The risk can be particularly high when the work is data-driven which can quickly resemble data-dredging. For more on this point, see our last two bad smells (*Not enough theory* and *Not much theory*).

An antidote and powerful way to consider practical significance is with effect size [33] especially if this can be couched in terms that are meaningful to the target audi-

ence. For instance, reducing unit test costs by $x\%$ or $y\%$ is inherently more engaging than the associated p value of some statistical test is below an arbitrary threshold. The notion of smallest effect size of interest has also been proposed as a mechanism to maintain focus on interesting empirical results [64] a point we shall return to.

4.2. Not Using Related Work

This issue should not exist. Nevertheless, we know of many articles that fail to make adequate use of related work. Typically we see the following specific sub-classes of “bad smell”.

The literature review contains no or almost no articles less than ten years old. Naturally older papers can be very important— but the absence of any recent contribution would generally be surprising. This strongly suggests the research is not state of the art, or is not being compared with state of the art.

The review overlooks relevant, recent systematic literature review(s). This strongly suggests that there will be gaps or misconceptions in the way the state of the art is represented. It may also allow bias to creep in since the authors’ previous work may become unduly influential.

The review is superficial, even perfunctory. Such an approach might be caricatured along the lines of:

“Here is some related work [1, ..., 30]. Now to get on with the important task of my describing own work.”

这种写作风格——审阅他人作品只是一种负担——使得文章之间的联系不太可能被看到，文章对知识体系的贡献也更少，但相反，它增加了确认偏差的脆弱性。[82]。由于对其他作品的解读很少甚至没有，这对读者也没什么帮助。哪些内容有用？哪些内容已经被取代了？

该评论是彻底的、相关的和公正的，但是，它并没有用于激发新的研究。换句话说，综述与新研究之间存在脱节。新结果与当前的最新研究成果无关。其结果是读者无法确定做出了哪些新的贡献。这也可能意味着错过了合适的基准。

研究方法的方面受到极其简单或过时的来源的推动，例如使用传统的统计文本。使用此类来源的众多危险之一是支持过时或不适当的数据分析技术或不再具有竞争力的学习算法。

一个建设性的建议是，特别是在没有相关系统文献综述的情况下，采用轻量级文献检查这些不必像完整的系统性文献综述那样复杂（可能需要数月才能完成）。然而，随着已发表的系统性文献综述数量的不断增长，其他研究人员可能已经至少部分地研究过与您正在探索的研究主题相关的文献。

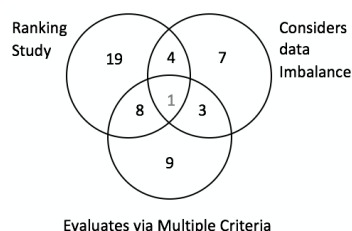


图 1：表格摘要3。

表 3: 21 篇被引用次数较高的软件缺陷预测研究, 按如下方式选取: (a) 从 Google Scholar 中搜索过去十年的“软件”和“缺陷预测”以及“OO”和“CK”; (b) 删除自出版以来每年被引用次数少于 10 次的任何内容; (c) 在得到的 21 篇文章中搜索 Agrawal 等人探索的三个研究项目, 即, 它们是否对多个分类器进行排名; 分类器是否按多个标准进行排名; 研究是否考虑类别不平衡。

参考	年	引用	排名 分类器?	已评估 使用 多种的 标准?	经过考虑的 数据 不平衡?
(一个)	2007	855	3	2	7
(二)	2008	607	3	1	7
(三)	2008	298	3	2	7
(四)	2010	178	3	3	7
(e)	2008	159	3	1	7
(六)	2011	153	3	2	7
(克)	2013	150	3	1	7
(八)	2008	133	3	1	7
(我)	2013	115	3	1	3
(j)	2009	92	3	1	7
(千)	2012	79	3	2	7
(左)	2007	73	7	2	3
(米)	2007	66	7	1	3
(名词)	2009	62	3	3	7
(哦)	2010	60	3	1	3
(页)	2015	53	3	1	7
(问)	2008	41	3	1	7
(右)	2016	31	3	1	7
(s)	2015	二十七	7	2	3
(吨)	2012	23	7	1	3
(你)	2016	15	3	1	7

轻量级文献检查示例

系统蒸发散, 见图1和表3来自 Agrawal 和 Menzies [2]。研究人员发现, 在过去十年中, 引用次数最多的软件缺陷预测研究中, 有二十多篇文章存在研究空白, 这些文章每年至少被引用10次。具体来说, 如图所示1大多数文章都避免了数据不平衡和通过多个标准进行评分的问题。当然, 研究空白需要引人入胜 (参见“坏味道”#1)。这一发现促使 Agrawal 和 Menzies 进行了一组独特的实验, 并最终发表了一篇 ICSE'18 论文。

4.3. 使用已弃用或可疑的数据

Researchers adopting a heavily data-driven approach, as is the case for software analytics, need to be vigilant regarding data quality. De Veaux and Hand point out “[e]ven when the statistical procedure and motives are correct, bad data can produce results that have no validity at all” [26]. A series of systematic reviews and mapping studies [68, 13, 85, 69] all point to problems of data quality not being taken very seriously and problems with how we definitively establish the validity of data sets, particularly when the gap in time and perhaps geography, between the collection and analysis is wide.

In terms of “smells” or warning signs we suggest researchers be concerned by the following:

- Very old data; e.g., the old PROMISE data from last-century NASA project (JM1, CM1, KC1, KC2, ...) or the original COCOMO data sets containing size and effort data from the 1970s.

- Problematic data; e.g., data sets with known quality issues (which, again, includes the old PROMISE data from last-century NASA project [94]).
- Data from trivial problems. For example, in the history of software test research, the Siemens test suite was an important early landmark in creating reproducible research problems. Inspired by that early work, other researchers have taken to document other, more complex problems³. Hence, a new paper basing all its conclusions on the triangle inequality problem from the Siemens suite is deprecated. Nevertheless tiny problems occasionally offer certain benefits, e.g., for the purposes of explaining and debugging algorithms, prior to attempting scale up.

That said, our focus is empirical studies in the field of software engineering where scale is generally the issue. In our experience, readers are increasingly interested in results from more realistic sources, rather than results from synthetic or toy problems. Given there now exist many on-line repositories where researchers can access a wide range of data from many projects (e.g. GitHub, GitTorrent⁴; and others⁵) this should not be problematic. Consequently, it is now normal to publish using data extracted from dozens to hundreds of projects e.g., [61, 3].

Finally, we acknowledge there are occasions when one must compromise, otherwise important and interesting software engineering questions may never be tackled. Not all areas have large, well-curated, high quality data sets. In such circumstances research might be seen as more exploratory or to serve some motivational role in the collection of better data. As long as these issues are transparent we see no harm.

4.4. Inadequate Reporting

Here there are two dimensions: completeness and clarity. Problems may be indicated by the following “smells”.

1. The work cannot be reproduced because the descriptions are too incomplete or high level. In addition, not all materials (data and code) are available [41, 79].
2. The work cannot be *exactly* 由于某些算法是随机的，且未提供种子，因此无法复制。典型示例是机器学习中用于交叉验证的折叠生成以及启发式搜索算法。
3. 不显著和事后“无趣”的结果不会被报告。这种看似无害的行为被称为报告偏差，它可能导致对效应大小的严重高估，并损害我们通过荟萃分析或类似方法构建知识体系的能力。41, 104]。
4. 实验细节中充斥着参数和实验设计的“神奇”数字或任意值。否则，重要的是要证明选择的合理性

³例如，请参阅软件工件基础设施存储库sir.unl.edu/portal。

⁴github.com/cjb/GitTorrent
⁵stiny.cc/seacraft

研究人员存在偏见和过度拟合的风险，例如，选择有利于他们所钟爱的算法的数值。标准的理由包括“通过标准差距统计程序进行选择”或进行一些敏感性分析来探索选择不同值的影响[87]。

5. 读者如果不通读全文，就不可能理解文章的主旨。结构化的摘要（例如本文所述）可以极大地帮助人类读者理解文章内容。[14，59，11] 和文本挖掘算法开始被部署来协助元分析，例如，[104]。
6. 使用极具特色的章节标题或文章组织结构。通常以下内容即可：引言、动机、相关工作、方法（分为数据、算法、实验方法和统计程序），然后是讨论威胁、效能威胁、结论、未来工作和参考文献。简单的表格总结算法细节、超参数等信息非常有用。
7. 文章使用了太多首字母缩略词。首次使用时，请定义术语。如果缩略词太多，请考虑使用词汇表。

我们用一些简单且有建设性的建议来结束本小节。

*努力获得可重复的结果：*桌子4定义了研究成果和结果的连续统一体。报告可重复的结果（位于该连续统一体的右侧）是理想情况，但这可能需要付出大量努力。关于如何构建研究成果以提高可重复性的实用讨论，请参阅“科学计算的足够好实践”。goo.gl/TD7Cax。

（题外话：对提高可重复性的要求必须从务实的角度进行评估。如果之前的研究使用的工具不是开源的，或者使用起来非常复杂的工具，那么要求后续工作完全重复之前的成果是不合适的。）

*考虑在每个表格或图片标题中添加摘要文本*例如，“图 X：统计结果表明，只有少数学习者能够成功完成这项任务”：审阅者通常会先浏览文章，只看图片。对于这样的读者来说，这些注释的局部性可能非常有帮助。

*考虑使用研究问题：*软件分析研究文章通常会自然地细分为几个研究问题。确定这些研究问题可能需要一些创造力，但却是一种强大的结构化工具。最好是一定的进展，例如，以下是来自[三十七]（讨论了调整数据挖掘算法超参数的优点）：

- RQ1：调整是否会提高预测器的性能分数？
- RQ2：调整是否会改变关于哪些学习者比其他学习者更优秀的结论？
- RQ3：调整是否会改变关于软件工程中哪些因素最重要这一结论？
- RQ4：调优是否慢得不切实际？

表 4：工件连续体从左到右流动。一旦作者应用了更多左侧的方法，更多文章将被归类到右侧。此表列出了 ACM 定义的徽章和定义（参见goo.gl/wVEZGx）。更广泛的社区可以将这些定义用作（1）理想目标，或（2）审核过去研究的方法，或（3）在会议或期刊上组织“工件轨迹”的指导。

不徽章	文物			结果	
	功能	可重复使用	可用的	已复制	转载
	 文物记录在案，持续的，完全的，可执行文件，以及包括合适的证据确认和验证。	 功能性+非常小心记录在案，结构良好重复使用和重新利用的程度促进。在特定规范以及研究界的标准对于这个文物类型严格坚持。	 功能性+放置在可公开访问档案库。DOI（数字对象标识符）或链接到此存储库具有唯一标识符提供对象的 DOI 信息。获取此类 DOI 很容易：只需将您的工件托管于存储库类似海上交通工具tiny.cc/seacraft。	 可用 + 主要结果文章已获得在随后的研究个人或团队除了作者使用，在某种程度上，文物提供方作者。	 可用 + 主要结果文章有到过独立地在获得随后的研究个人或团队除了作者，无需使用作者供应文物。

- RQ5：调音容易吗？
 - RQ6：数据挖掘器是否应该使用其默认调整的“现成”版本？
- 有时，与研究问题相关的一种有用的排版技巧是使用结果框来强调问题的答案。以下是 Zimmermann 等人早期的一个示例。[114]，2004. 尽管此类摘要可能很有影响力，但需要精心构建，以避免过度简化。

*One can either have **precise** suggestions or **many** suggestions, but not both.*

4.5. 动力不足的研究

分析的功效是指检验拒绝假零假设（即正确报告非零效应）的概率。更通俗地说，它是指在存在效应的情况下，发现效应的能力。功效是 β 或犯 II 类错误（未能检测到支持零假设的真实效应，即假阴性）的概率，因此功效 = $1 - \beta$ 然而，这需要权衡犯第一类错误（即假阳性）的概率。在极端情况下，我们可以通过设置极高的接受阈值来防止假阳性，但结果会导致假阴性率升高。当然反之亦然。

那么应该怎么做 β 应该设置为多少？传统上，人们认为功率应该为 0.8 [21] 表明研究人员可能会考虑 I 类错误（错误地拒绝

零假设（实际上没有影响）比第二类错误问题严重四倍，因此需要相应更多的检验。因此，如果显著性水平设定为 $\alpha=0.05$ 那么 $\beta=4\alpha=0.2$ 因此，按照要求，我们的力量是 $1-\beta=0.8$ 。重申一下，重要的一点是，我们必须在 I 型错误和 II 型错误之间进行权衡，并且我们的偏好有些随意。

然而，我们希望研究的力量是什么，以及它实际上这是两码事。有四个因素会影响功效，分别是：（i）样本量，（ii）待研究效应的未知真实大小，（iii）所选的 α 以及（iv）所选 β [22, 33]此外，实验设计可以提高功效，例如通过测量实验单元内的变化（例如，通过重复测量设计），而不是单元之间的变化[75]。

许多评论员建议，应确定拟议实验或调查的力量先验[30]其目的是检验实验是否有合理的机会检测到实际存在的效应。一个显而易见且直接的困难是估计待研究的效应大小[74]。更糟糕的是，有令人信服的证据表明，由于报告和出版偏见，我们系统地高估了效应大小。[15, 53]然而，效力问题不仅仅是因为未能检测到实际存在的效应而浪费科学资源。还有另外两个严重的负面后果。首先，倾向于高估所测量的效应大小；其次，出现虚假统计显著性发现的可能性较高。[50, 23]确实，毫不夸张地说，低效和动力不足的研究是对元分析和构建有意义的软件工程知识体系的最大威胁[15, 104]。

讽刺的是，软件分析研究在大型、复杂且嘈杂的数据集中能够揭示模式和效应，而这恰恰是其失败之处。再加上缺乏指导理论（这使得人们难以区分“赢家诅咒”和一些实质性的影响），[6]、一致的分析方法和过多的研究者自由度[95, 71]导致研究效力低下，并且很可能高估效应大小，而其他研究人员很难复制[50, 93, 53]作为解决方案，我们强烈建议：

1. 提前确定感兴趣的最小效应大小（SESOI）[64] 这可能需要与问题所有者（通常是从从业者）进行辩论。这也有助于解决我们的“第一大恶臭”，因为它可以减少进行无人关注的研究的可能性。
2. 此外，应提前进行功效计算，以确定研究得出预期规模效应的可能性。如果可能，在有相关先前研究的情况下，使用偏差校正后的合并估计值[90]以帮助解决可能被夸大的估计。
3. 如果功效较低，则修改研究设计[75]，使用更大的样本或

⁶Huang等人[44]描述了基因组学中的类似情况，其中可能发现数百万个关联，但缺乏理论意味着对研究人员的指导很少。

调查另一个问题。

与此类问题相关的“气味”可能出奇地微妙。然而，一个线索可能是类似这样的论点：

尽管样本量很小，我们仍然能够以非常低的 α 。因此，可以得出结论，考虑到检测的固有困难，这种影响肯定比最初看起来的还要强烈。

不幸的是，事实并非如此。如果研究效力不足，可能是由于样本量较小，那么结果极有可能是假阳性或 I 型错误。⁷¹如果研究的效力低，那么这就是一个问题，而且这个问题不能通过寻找小样本来解决。 α 值。小样本带来的问题是，变异性会很高，只有偶然获得足够低 p 值的研究——因此满足接受标准——通常 $\alpha=0.05$ ——这些必然会夸大对效应的估计。这有时被称为“赢家的诅咒”。达到统计显著性的观测效应实际上反映真实效应的概率可以计算为 $([1-\beta] \times R)/([1-\beta] \times R + \alpha)$ 在哪里 $(1-\beta)$ 是力量， β 是第二类错误， α 是 I 型错误， R 是研究前概率，即在所有被研究的效应中，某个被研究的效应真正非零。) [71, 104] 注意，这个问题会导致下一个“坏味道”。

4.6. $p < 0.05$ 以及所有这一切!

尽管被广泛弃用[17, 96, 23]，零假设显著性检验 (NHST) 仍然是统计分析的主导方法。因此，以下虚构的⁷例如“味道不好”：

我们的分析得出了统计学上显著的相关性($r=0.01$; $n=18000$; $\alpha=0.049$; $\alpha=0.05$)。

和

表 X 总结了应用 ABC 推理测试的 500 个结果，并表明在惯常阈值 α 下，40 个结果具有统计显著性 $=0.05$ 。

首先，低（并且在 $\alpha=0.05$ （I 类错误的接受阈值） α 值不应与现实世界的实际效果相混淆。相关性 $r=0.01$ 微不足道，实际上与无相关性无异。它只是样本量过大造成的结果（ $n=18000$ ）NHST 的逻辑要求将经验结果二分为真（存在非零效应）和假（效应大小恰好为零）。

因此，方法论者鼓励感兴趣的最小效应量（SESOI）。应指定前这项研究也具有

⁷由于人们普遍认为 p 值的使用存在问题 [96, 53] 我们认为单挑个人是令人反感的。

让研究人员重新参与到他们试图解决的软件工程问题中。例如，对于软件工作量预测，在与问题所有者协商后，我们可以得出结论，SESOI 能够将平均误差降低 25%。通常，虽然并非必须，效应大小会以标准化度量来表示，以便于在不同设置之间进行比较，因此上述 SESOI 可以通过观测值的变异性（标准差）进行归一化。有关易于理解的概述，请参阅 [33, 22] 如果文章报告的效应大小带有置信限度，对实践者来说会很有帮助。毕竟，我们经历的效应并非无价值！

第二个“坏味道”的例子，问题在于进行过多的统计检验，而只关注那些“显著”的检验。假设进行了 500 次检验，我们接受错误拒绝原假设的概率为 1/20，那么仅凭随机数据，我们预计就会观察到 40 个“阳性”结果！一系列修正措施已被提出[7]使用 Benjamini-Hochberg 方法控制错误发现率[8] 可能是合适的，因为它可以纠正接受阈值，但不像传统的 Bonferroni 校正那样严格（ α/t 在哪里 t 是正在进行的测试数量）。

避免过度统计检验的一种方法是在进行统计分析之前对结果进行聚类。在这种方法中，我们不是说“哪些分布不同？”，而是在应用统计信息之前应用一些分组运算符。例如，考虑一下 Mittas 和 Angelis 推荐的 Scott-Knott 程序[78] 该方法对 $升$ 治疗 IS 测量结果按其中位数得分进行排序。然后拆分 $升$ 进入子列表 m, n 目的是最大化分组前后观察到的表现差异的预期值。斯科特-诺特程序会按如下方式确定其中一个分组是“最佳”。对于列表长、中、短尺寸秒、毫秒、纳秒在哪里 $升=米 \cup n$ ，“最佳”划分最大化 $埃(\Delta)$ ；即分割前后预期均值的差异：

$$埃(\Delta) = \frac{\text{多发性硬化症}}{IS} \text{腹肌}(米.\mu - 升.\mu)^2 + \frac{\text{纳秒}}{IS} \text{腹肌}(n.\mu - l.\mu)^2$$

斯科特-诺特算法随后会检查“最佳”划分是否真的有用。为了确定这一点，该程序会应用一些统计假设检验 $哈$ 检查是否 m, n 存在显著差异。如果存在显著差异，Scott-Knott 算法便会对“最佳”划分的每一半进行递归。

对于具体示例，请考虑以下结果 $升=5$ 治疗方法：

rx1 = [0.34, 0.49, 0.51, 0.6] rx2 = [0.6, 0.7, 0.8, 0.9] rx3 = [0.15, 0.25, 0.4, 0.35] rx4 = [0.6, 0.7, 0.8, 0.9] rx5 = [0.1, 0.2, 0.3, 0.4]

经过排序和划分后，Scott-Knott 确定：

- 排名第一的是 rx5，中位数 = 0.25
- 排名第一的是 rx3，中位数 = 0.3
- 排名第二的是 rx1，中位数为 0.5

治疗	多次运行的结果							
无SWReg	174	三十八	0	32	50	141	三十四	114
无SLReg	127	73	45	三十五	四十七	159	三十七	103
无CART-On	119	257	425	294	181	98	409	520
无CART关闭	119	257	425	294	181	98	409	520
无PLSR	63	213	338	358	163	四十四	14	520
无PCR	55	240	392	418	176	45	12	648
无1NN	119	66	81	82	70	110	118	122
对数SWReg	109	23	9	7	52	100	12	164
对数SLReg	108	61	49	53	55	156	二十八	183
对数CART-On	119	257	425	294	181	98	409	520
对数CART关闭	119	257	425	294	181	98	409	520
对数PLSR	163	7	60	70	2	152	31	310
对数PCR	212	180	145	155	70	68	1	553
对数-1NN	46	231	246	154	46	23	200	197

注意：治疗有一个预处理器（“no”=无预处理器；“log”=使用对数变换）和一个学习器（例如，“SWReg”=逐步回归；CART=决策树学习器，在构建树后对其叶子进行后修剪）。

图 2：如何避免数据呈现（本例中，第一列的处理方法应用于多个随机选择的数据分层，并估算百分比误差）。请注意：(a) 没有按最有趣到最不有趣的顺序排列处理方法；(b) 没有对分布进行汇总；(c) 没有指示哪些值是最佳值；(d) 没有将统计上难以区分的结果聚类在一起。有关更好地呈现此类数据的方法，请参见图3。

- 排名第三的是 rx2，中位数为 0.75
- 排名第三的是 rx4，中位数为 0.75

请注意，Scott-Knott 发现 rx5 和 rx3 之间几乎没有显著差异。因此，即使它们的中位数不同，它们的秩也相同。有关此类分析的更大示例，请参见图2&3。

Scott-Knott 程序可以改善过度统计检验的问题。为了说明这一点，我们考虑一个包含 10 个治疗方案的全对假设检验。这些检验结果可以在统计学上进行比较（ 10_2-10 ）/2 = 45 方法。为了大幅减少成对比较的次数，Scott-Knott 仅调用假设检验后它找到了最大化性能差异的分割方法。因此，假设对整个数据进行二元分割，并且所有分割结果都不同，那么 Scott-Knott 算法将被称为日志₂（10）≈ 3 times. A 95% confidence test run for these splits using Scott-Knott would have a confidence threshold of: $0.95_3 = 86\%$ assuming no corrections are made for multiple tests..

Implementations that use bootstrapping and a non-parametric effect size test (Cliff's Delta) to control its recursive bi-clustering are available as open source package, see [goo.gl/uoj3FL](https://github.com/3FL/3FL). That package converts data such as Figure 2 into reports like Figure 3.

4.7. Assumptions of normality and equal variances in statistical analysis

A potential warning sign or “bad smell” is when choosing statistical methods, to overlook the distribution of data and variances and focus on the “important” matters of description e.g., using measures of location such as the mean, and the application of inferential statistics such as t-tests.

It is now widely appreciated that applying traditional parametric statistics to data that violates the assumptions of normality, i.e., they deviate strongly from a Gaussian

RANK	TREATMENT	MEDIAN	(75-25)TH			
1	no-SWReg	50	107	-o	-----	
1	log-SWReg	52	97	o	--	
1	log-SLReg	61	55	-o	----	
1	log-PLSR	70	132	-o	-----	
1	no-SLReg	73	82	-o	-----	
1	no-1NN	92	37		o	
2	log-PCR	155	142	---	o	-----
2	log-1NN	158	154	-	o	--
2	no-PLSR	163	294	--	o	-----
2	no-PCR	176	337	--	o	-----
3	log-CART-Off	257	290	-----	o	-----
3	log-CART-On	257	290	-----	o	-----
3	no-CART-On	257	290	-----	o	-----
3	no-CART-Off	257	290	-----	o	-----

Figure 3: Using Scott-Knott, there is a better way to representation the error results see in Figure 2 (and *lower* error values are *better*). Note that, here, it is clear that the “SWReg”, “CART” treatments are performing best and worst (respectively). Results from each treatment sorted by their median (and the first rows with less error are better than the last rows with most error). Results presented numerically and visually (right-hand column marks the 10th, 30th, 50th, 70th, 90th percentile range; and ‘o’ denotes the median value). Results are clustered in column one using a Scott-Knott analysis. Rows have different ranks if Cliff’s Delta reports more than a small effect difference and a bootstrap test reports significant differences. A background gray share is used to make different ranks stand out.

distribution *can* lead to highly misleading results [58]. However, four points may be less appreciated within the software engineering community.

1. Even minor deviations can have major impacts, particularly for small samples [110]. These problems are not always easily detected with traditional tests such as Kolmogorov-Smirnov so visualization via qq-plots should be considered.
2. Differences between variances of the samples or heteroscedasticity, may be a good deal more problematic than non-normality [34]. Thus the widespread belief that t-tests are robust to departures from normality when sample sizes are approximately equal and >25 is unsafe [34, 113].
3. Although different transformations, e.g., log and square root can reduce problems of asymmetry and outliers they may well be insufficient to deal with other assumption violations.
4. 最后，过去30年来，稳健统计领域取得了重大进展，这些进展通常比非参数统计更强大、更受欢迎。这包括修剪和缩尾操作[46, 57]。

由于SE数据集可能高度偏斜、重尾或两者兼而有之，我们建议进行稳健检验以进行统计推断。需要注意的是，我们认为（尽可能）应优先使用稳健检验，而非基于秩的非参数统计数据。后者，例如Wilcoxon符号秩检验程序，可能缺乏检验功效，并且在出现许多平局时存在问题[10Kitchenham等人[57] 在软件工程的背景下提供有用的建议，以及 Erceg-Hurn 和 Mirosevich 提供的易于理解的概述 [三十四有关广泛而详细的处理，请参阅[110]。

另一种方法是使用非参数引导法（参见 Efron 和 Tibshirani [三十二，p220-223]）。请注意图3是通过这样的引导测试生成的。

随着强大计算机的出现，蒙特卡罗和随机化技术可以成为参数方法非常实用的替代方案。[73](#)]

4.8. 没有数据可视化

虽然通常用一些统计量来描述数据，例如均值、中位数和四分位数，但这并不一定能检验异常模式和异常现象。因此，我们强烈建议[数据可视化](#)之前 *应用统计检验*。例如：

- 图的水平箱线图[3](#)可以在一行中呈现大量样本的结果。通过对数据的目视检查，分析师可以检查他们的结论是否由于极端异常值或高方差等原因而站不住脚。
- 再举一个例子，考虑图[4](#)显示了探索 SMOTE 价值的 5*5 交叉验证实验的结果：
 - SMOTE 是一种数据再平衡技术，它可以调整训练数据，使任何类别都不再是少数。[\[2\]](#)。
 - 5*5 交叉验证实验是一种评估技术，其中 m 次，我们随机化数据的顺序。每次将项目数据分成 k 个垃圾箱，继续训练 $N - k$ 箱体，对保留值进行测试（对所有箱体重复此操作）。此过程产生 25 个结果，如图所示[4](#)以中位数和四分位数间距（IQR）的形式报告。

通过这种直观的观察，在进行任何统计之前，我们可以观察到，在任何情况下，使用 SMOTE 都不会比不使用 SMOTE 更糟糕。此外，如图右侧结果所示[4](#)，有时应用 SMOTE 会带来很大的改进。

4.9. 不探索稳定性

由于多种原因，软件工程数据集经常表现出很大的差异[\[77\]](#)因此，检查你想要报告的效应是否真的存在，或者仅仅是噪音造成的，这一点很重要。因此：

- 当有多个数据集可用时，尝试多个数据集。
- 报告集中趋势的测量值（例如中位数）和离散度（例如标准差或四分位距（IQR）（第 75 到第 25 百分位数））。
- 不要仅仅对 N 个数据集的结果进行平均。相反，要总结[传播](#)数据集的结果可能包括图形的创造性使用，例如参见图[4](#)上图。回想一下，图中底部的虚线表示的是图中每个数据集 25 个结果的变异性。这里的关键点在于，这种变异性非常小；也就是说，中位数结果之间的差异通常比随机选择训练集所引入的变异性要大得多（当然，这种视觉印象需要理性的判断）。



图 4: 在 25 个 SE 数据集上进行文本挖掘时使用 SMOTE 的效果。图中, x 轴上的数据集按使用和未使用 SMOTE 的差异大小排序。中位数和 IQR 分别以实线和虚线表示 (中位数是 50% 百分位数的结果, IQR 是 (75-25)% 的百分位数)。来自 [63]。

检查——这就是统计检验的作用)。如果没有离散统计数据, 元分析将极其困难, 因为需要构建置信区间才能得出结果。[12, 90]。

- 如果仅探索一个数据集, 则重复分析大约 20 到 30 次 (然后使用引导程序 (即, 有放回抽样) 探索以这种方式生成的分布)。
- 时间验证: 将给定的数据划分为多个时间序列 (例如, 同一项目的不同版本), 然后使用过去的数据进行训练, 并使用未来的数据进行测试。我们建议保持未来测试集的大小不变; 例如, 使用软件版本进行训练 $SE_{-2,-1}$, 然后在版本上测试 SE 。这种验证增强了真实性, 并且可能会引起研究目标用户更浓厚的兴趣。
- 跨项目验证: 使用另一个项目的数据进行训练, 然后在该项目上进行测试。需要注意的是, 当项目使用具有不同属性名称或特征的数据时, 这种跨项目迁移学习可能会变得有些复杂。
- 项目内验证 (也称为 “交叉验证”) : 将项目数据划分为 N 垃圾箱, 继续训练 $N-1$ 箱体, 在留出的位置进行测试。对所有箱体重复此操作。

请注意, 由于目标始终固定, 时间验证 (以及留一法交叉验证 LOOCV) 不会产生结果范围。然而, 交叉验证和组内验证会产生结果分布, 因此必须进行适当的分析, 以便报告集中趋势 (例如平均值或中位数) 和离散度 (例如方差或 IQR) 的度量。通常, 为了获得 25 个跨项目学习样本, 需要随机选择 90% 的数据 25 次。为了

获得例如 25 个项目内学习样本， $m=5$ 次对数据的顺序进行打乱，并针对每次打乱运行 $k=5$ 交叉验证。警告：对于少于 60 个项目的小型数据集，请使用 $M, N=8, 3$ (自从 $8 \times 3 \approx 25$)。

4.10. 未调整

许多机器学习者，包括数据挖掘者，都选择了错误的默认设置[108, 45, 三十七, 105, 1, 2]。Numerous recent results show that the default control settings for data miners are far from optimal. For example, when performing defect prediction, various groups report that tuning can find new settings that dramatically improve the performance of the learned mode [37, 105, 106]. As a specific of that tuning, consider the standard method for computing the distance between rows x and y (where each row contains i variables):

$$d(x, y) = \left(\sum_i (x_i - y_i)^2 \right)^{1/2}$$

When using SMOTE to re-balance data classes, tuning found it useful to alter the “2” value in the above equation to some other value (often, some number as high as “3”).

One particularly egregious “smell” is to compare a heavily tuned new or pet approach with other benchmarks that are untuned. Although, superficially the new approach may appear far superior what we really learn is using the new technique well can outperform using another technique badly. This kind of finding is not particularly useful.

As to how to tune, we *strongly* recommend against the “grid search” i.e., a set of nested for-loops that try a wide range of settings for all tuneable parameters. This is a slow approach; an exploration of grid search for defect prediction and found it took days to terminate [37]. Not only that, it has been reported that grid search can miss important optimizations [38]. Every grid has “gaps” between each grid division which means that a supposedly rigorous grid search can still miss important configurations [9]. Bergstra and Bengio [9] comment that for most data sets only a few of the tuning parameters really matter – which means that much of the run-time associated with grid search is actually wasted. Worse still, Bergstra and Bengio observed that the important tunings are different for different data sets, a phenomenon that makes grid search a poor choice for configuring software analytics algorithms for new data sets.

Rather than using “grid search”, we recommend very simple optimizers (such as differential evolution [103]) can suffice for finding better tunings. Such optimizers are very easy to code from scratch. They are also readily available in open-source toolkits such as jMETAL⁸ or DEAP⁹.

4.11. Not Exploring Simplicity

One interesting, and humbling, aspect of using Scott-Knott tests is that, often, dozens of treatments can be grouped together into just a few ranks. For example, in the Scott-Knott results of [40], 32 learners were divided into only four groups.

⁸jmetal.sourceforge.net

⁹github.com/DEAP/deap

While not all problems succumb to simple approaches, it is wise to compare a seemingly sophisticated method against some simpler alternative. Often when we code and test the seemingly naïve method, we find it has some redeeming feature that makes us abandon the more complex methods [19, 36, 62]. Moreover, the more parsimonious our models, the less prone to overfitting [43].

这种探索简单性的诉求在当前的研究氛围中尤为重要，因为当前的研究氛围往往侧重于深度学习，而深度学习确实是一种非常缓慢的方法。三十五深度学习本质上更适合解决具有高度复杂内部结构的问题。66]。然而，许多SE数据集的规则结构非常简单，因此非常简单的技术可以执行得更快，并且性能与深度学习相当，甚至更好。这种更快的执行速度非常有用，因为它们可以更轻松地复制先前的结果。

这并不是说深度学习与 SE 无关。相反，这意味着任何深度学习（或任何其他复杂技术）的结果也应该与一些更简单的基线进行比较（例如，[三十六]）来证明额外的复杂性是合理的。例如，Whigham 等人[109] 提出了一个基于自动数据转换和 OLS 回归的简单基准作为比较器，其想法是，如果复杂的方法不能胜过简单的方法，人们就不得不质疑它的实用价值。

探索更简单选项的三种简单方法是 *侦察兵*，*随机搜索* 和 *特征选择*。霍尔特 [四十七] 报告称，一个名为 1R 的非常简单的学习器可以 *侦察* 在数据集中领先一步，并汇报是否需要更复杂的学习。更一般地说，通过首先运行一个非常简单的算法，很容易快速获得基线结果，从而了解问题的整体结构。

至于随机搜索，为了优化，我们经常发现 *随机搜索*（例如，利用差异进化[103]）在解决各种问题方面表现非常出色[三十七，1，2]。请注意，Holte 可能会说，对问题的初始随机搜索是 *侦察* 那个问题。

此外，对于分类，我们发现 *特征选择* 通常会发现大多数属性都是多余的，可以丢弃。例如，[76] 表明，在保留预测能力的同时，最多可以丢弃 41 个静态代码属性中的 39 个。[20]。这一点很重要，因为从较少特征中学习到的模型更容易阅读、理解、批判，也更容易向业务用户解释。此外，通过去除虚假特征，研究人员得出虚假结论的可能性也会降低。此外，降低数据维度后，任何后续处理都会更快、更容易。

请注意，许多文章都执行非常简单的线性时间特征选择，例如 (i) 按相关性、方差或熵排序；(ii) 然后移除剩余的最差特征；(iii) 从剩余特征构建模型；(iv) 如果新模型比之前学习到的模型更差，则停止；(v) 否则，返回步骤 (ii)。Hall 和 Holmes 报告称，这种贪婪的搜索算法可以提前停止，并且通过并行增加有用特征集可以获得更好的结果（参见他们基于相关性的特征子集选择算法，该算法在 [四十二]）。

在继续之前，我们注意到简单性研究的一个矛盾之处——它可能非常难以进行。在证明某种更简单的方法比最简单的方法更好之前，

表 5: Ghotra 等人[40] 来自 ICSE'15 的文章使用 Scott-Knott 程序对 32 个常用于缺陷预测的不同学习器进行了排名。这些排名分为四组。下表显示了这些组的示例。

秩	学习者	笔记
1个“最佳”	射频=随机森林	基于熵的决策树的随机森林。
	LR=逻辑回归	广义线性回归模型。
2	KNN 第 k 个最近邻	通过查找对新实例进行分类子类似情况的例子。Ghotra 等人建议 $k=8$ 。
	NB=朴素贝叶斯	通过以下方式对新实例进行分类：（a）收集不同类别的旧实例中属性的平均值和标准差；（b）返回属性在统计上与新实例最相似的类。
3	DT=决策树	通过选择减少类分布熵的属性分割来递归地划分数据。
4个“最差”	支持向量机=支持向量机	将原始数据映射到更高维的空间中，以便更容易区分示例。

如果要用复杂的、最先进的替代方案来测试，可能需要先用简单的方案来测试复杂的方案。这可能会导致一种特殊的情况：需要耗费数周的 CPU 来评估（比如）某个非常简单的随机方法的值，而该方法只需几分钟就能终止。

4.12. 没有证明学习者的选择

这种风险和“坏味道”源于机器学习技术的快速发展，以及构建不同学习器复杂集成的能力，这意味着排列的数量正在迅速变得巨大。对一个旧的学习器进行排列很容易左进入左每个新的学习器都会产生一篇新的文章。“坏味道”在于这样做没有动机，也没有先验的理由来解释我们为什么会期望左有趣且值得探索。

When assessing some new method, researchers need to compare their methods across a range of interesting algorithms. Comparing all new methods to all prior methods is a daunting task (since the space of all prior methods is so very large). Instead, we suggest that most researchers maintain a watching brief on the analytics literature looking for prominent papers that cluster together learners that appear to have similar performance, then draw their comparison set from at least one item of each cluster.

For learners that predict for discrete class learning, one such clustering can be seen in Table 9 of [40]. This is a clustering of 32 learners commonly used for defect prediction. The results clusters into four groups, best, next, next, worst (shown in Table 5). We would recommend:

- Selecting your range of comparison algorithms as one (selected at random) for each group;
- Or selecting your comparison algorithms all from the top group.

For learners that predict for continuous classes, it is not clear what is the canonical set, however, Random Forest regression trees and logistic regression are widely used.

For search-based SE problems, we recommend an experimentation technique called (you+two+next+dumb), defined as

- ‘You’ is your new method;
- ‘Two’ are well-established widely-used methods (often NSGA-II and SPEA2 [89]);
- A ‘next’ generation method e.g. MOEA/D [112], NSGA-III [27];
- and one ‘dumb’ baseline method (random search or SWAY [19]).

5. Discussion

Methodological debates are evidence of a community of researchers rechecking each other’s work, testing each other’s assumptions, and refining each other’s methods. In SE we think there are too few articles that step back from specific issues and explore the more general question of “what is a valid article?”.

To encourage that debate, this article has proposed the idea of a “bad smell” as a means to use surface symptoms to highlight potential underlying problems. To that end, we have identified over a dozen “smells”.

This article has argued that, each year, numerous research articles are rejected for violating the recommendations of this article. Many of those rejections are avoidable — some with very little effort — just by attending to the bad smells listed in this article. This point is particularly important for software practitioners. That group will struggle to get their voice heard in the research literature unless they follow the established norms of SE articles. To our shame, we note that those norms are rarely recorded — a problem that this article is attempting to solve.

We make no claim that our list of bad smells is exhaustive – and this is inevitable. For example, one issue we have not considered here is assessing the implication of using different evaluation criteria; or studying the generality of methods for software analytics. Recent studies [62] 的研究显示，看似最先进的技术（例如迁移学习）如果是为某个领域（缺陷预测）设计的，那么在应用于其他领域（例如代码异味检测等）时，效果实际上非常糟糕。尽管如此，本文的总体目标是鼓励更多关于“有效”文章的讨论，并认识到有效性并非一个非黑即白的概念（因此使用了引号）。我们希望在更广泛的研究领域中进行进一步的讨论，能够显著完善这份清单。

我们认为，可以按照以下方式开展此类辩论：

1. 多读书，多思考。研究软件分析及其他领域所使用的方法。阅读文章时，不仅要考虑论点的内容，还要考虑其形式。
2. 宣传和分享“坏味道”和反模式，作为分享不良实践的一种手段，但目的是宣传良好实践。
3. 预先注册研究并确定（实际）感兴趣的最小效应大小。

4. 提前分析研究的可能效力。如果效力过低，则应修改研究设计（例如，收集更多数据或进行类型内分析）或更改研究。
5. 努力获得可重复的研究结果。
6. 制定、同意并使用软件分析研究的社区标准。

关于最后一点，正如我们上面提到的，本文是制定软件分析社区标准战略计划的一部分。为此，我们希望它能够有所启发：

- 关于最佳和最差论文写作实践的更广泛的讨论；
- 一系列研讨会，研究人员齐聚一堂，讨论并宣传我们社区对什么是“好”论文的看法；
- 后续文章将扩展和改进上述指导。

致谢

我们非常感谢编辑和审稿人的详细和建设性的评论。

参考

参考

- [1] A. Agrawal, W. Fu, T. Menzies, 《主题建模出了什么问题？（以及如何使用基于搜索的 SE 修复它）》，CoRR abs/1608.08176, URL<http://arxiv.org/abs/1608.08176>。
- [2] A. Agrawal, T. Menzies, “更好的数据”比“更好的数据挖掘者”更好吗？论调整 SMOTE 对缺陷预测的好处，载于：第 40 届国际软件工程会议论文集，ACM, 1050–1061, 2018 年。
- [3] A. Agrawal, A. Rahman, R. Krishna, A. Sobran, T. Menzies, 《我们不需要另一个英雄？“英雄”对软件开发的影响》，CoRR abs/1710.09055, 网址<http://arxiv.org/abs/1710.09055>。
- [4] A. Arcuri, L. Briand, 《使用统计测试评估软件工程中的随机算法的实用指南》，载于：第 33 届国际软件工程会议论文集，ICSE '11, ACM, 纽约，纽约州，美国，ISBN 978-1-4503-0445-0, 1-10, 2011 年。
- [5] K. Beck, M. Fowler, G. Beck, 《代码中的坏味道》，《重构：改进现有代码的设计》（1999），75–88。
- [6] A. Begel, T. Zimmermann, 分析一下！软件工程数据科学家面临的 145 个问题，载于：第 36 届 ACM 国际软件工程会议论文集，ACM, 2014 年 12 月 23 日。

- [7] R. Bender, S. Lange, 调整多重检测——何时以及如何? , 临床流行病学杂志 54 (4) (2001) 343–349。
- [8] Y. Benjamini, D. Yekutieli, 《依赖性条件下的多重测试中错误发现率的控制》, 《统计年鉴》 29 (4) (2001) 1165–1188。
- [9] J. Bergstra, Y. Bengio, 超参数优化的随机搜索, 机器学习研究杂志 13 (2月) (2012) 281–305。
- [10] C. Blair、J. Higgins, 《不同总体形态下配对样本 t 检验与 Wilcoxon 符号秩检验的功效比较》, 《心理学公报》 97 (1) (1985) 119–128。
- [11] A. Booth、A. O'Rourke, 《结构化摘要在 MEDLINE 信息检索中的价值》, 《健康图书馆评论》 14 (3) (1997) 157–166。
- [12] M. Borenstein、L. Hedges、J. Higgins、H. Rothstein, 《元分析导论》, Wiley, 英国西萨塞克斯郡奇切斯特, 2009 年。
- [13] M. Bosu、S. MacDonell, 《经验软件工程中的数据质量: 有针对性的回顾》, 载于: 《第 17 届软件工程评估与评定国际会议论文集》, ACM, 171–176, 2013 年。
- [14] D. Budgen, A. Burn, B. Kitchenham, 通过结构化摘要报告计算项目: 一项准实验, 实证软件工程 16 (2) (2011) 244–277。
- [15] K. Button, J. Ioannidis, C. Mokrysz, B. Nosek, J. Flint, E. Robinson, M. Munafò, 电源故障: 为何小样本量会破坏神经科学的可靠性, 《自然评论神经科学》 14(5)(2013) 365。
- [16] J. Carpenter, J. Bithell, Bootstrap 置信区间: 何时, 哪个, 什么? 医学统计学家实用指南, 医学统计学 19(9) (2000) 1141–1164。
- [17] RP Carver, 《反对统计显著性检验的案例再探》, 《实验教育杂志》 61 (4) (1993) 287–292。
- [18] G. Cawley、N. Talbot, 《论模型选择中的过度拟合及随后的性能评估中的选择偏差》, 《机器学习研究杂志》 11(7)(2010) 2079–2107。
- [19] J. Chen, V. Nair, R. Krishna, T. Menzies, “采样”作为基于搜索的软件工程的基线优化器, IEEE 软件工程学报。
- [20] Z. Chen, T. Menzies, D. Port, D. Boehm, 为软件成本建模寻找正确的数据, IEEE 软件 22 (6) (2005) 38–46。
- [21] J. Cohen, 《行为科学的统计功效分析》, Lawrence Earlbaum Associates, 新泽西州希尔斯代尔, 第 2 版, 1988 年。

- [22] J. Cohen, 《权力入门》, 《心理学公报》 112 (1) (1992) 155–159。
- [23] D. Colquhoun, 对错误发现率和 p 值误解的调查, 皇家学会开放科学 1 (140216)。
- [24] R. Courtney, D. Gustafson, 软件度量中的散弹枪相关性, 软件工程杂志 8 (1) (1993) 5-13。
- [25] DS Cruzes, T. Dybå, 软件工程研究综合: 一项第三方研究, 信息与软件技术 53 (5) (2011) 440–455。
- [26] R. De Veaux, D. Hand, 《如何利用坏数据说谎》, 《统计科学》 20(3)(2005) 231—238。
- [27] K. Deb, H. Jain, 一种基于参考点非支配排序方法的进化多目标优化算法, 第一部分: 解决带框约束的问题, IEEE 进化计算汇刊 18(4) (2014) 577–601。
- [28] P. Devanbu, T. Zimmermann, C. Bird, 《经验软件工程中的信念与证据》, 载于: IEEE/ACM 第 38 届国际软件工程会议 (ICSE), IEEE, 108–119, 2016 年。
- [29] T. Dybå, T. Dingsøyr, 《软件工程系统评价中的证据强度》, 载于: 第二届 ACM-IEEE 国际经验软件工程与测量研讨会论文集, ACM, 178–187, 2008 年。
- [30] T. Dybå, V. Kampenes, D. Sjøberg, “软件工程实验统计功效的系统评价”, 信息与软件技术 48 (8) (2006) 745–755。
- [31] B. Earp, D. Trafimow, 《复制、证伪和社会心理学中的信任危机》, 《心理学前沿》 6 (2015) 621–6322。
- [32] B. Efron, R.J. Tibshirani, 《Bootstrap 简介》, Mono. Stat. Appl. Probab., Chapman and Hall, 伦敦, 1993 年。
- [33] P. Ellis, 《效应量基本指南: 统计功效、荟萃分析和研究结果的解释》, 剑桥大学出版社, 2010 年。
- [34] D. Erceg-Hurn, V. Mirosevich, 现代稳健统计方法: 一种最大化研究准确性和力量的简便方法, 美国心理学家 63 (7) (2008) 591–601。
- [35] W. Fu, T. Menzies, 易胜难: 深度学习案例研究, 载于: 2017 年第 11 届软件工程基础联合会议论文集, ACM, 49–60, 2017 年。
- [36] W. Fu, T. Menzies, 重新审视无监督学习在缺陷预测中的应用, 载于: 2017 年第 11 届软件工程基础联合会议论文集, ACM, 72–83, 2017 年。

- [37] W. Fu, T. Menzies, X. Shen, 《软件分析调优》，信息软件技术, 76 (C) (2016) 135–146, ISSN 0950-5849。
- [38] W. Fu、V. Nair、T. Menzies, 《为什么差分进化比网格搜索更适合调整缺陷预测器? 》, CoRR abs/1609.02613, URL<http://arxiv.org/abs/1609.02613>。
- [39] A. Gelman, J. Carlin, H. Stern, D. Dunson, A. Vehtari, D. Rubin, 《贝叶斯数据分析》，CRC, 第 3 版, 2013 年。
- [40] B. Ghotra、S. McIntosh、AE Hassan, 重新探讨分类技术对缺陷预测模型性能的影响, 第 37 届国际软件工程会议论文集第 1 卷, IEEE 出版社, 789–800, 2015 年。
- [41] S. Goodman, D. Fanelli, J. Ioannidis, 研究可重复性意味着什么? , Science Translational Medicine 8 (341) (2016) 341ps12–341ps12。
- [42] MA Hall、G. Holmes, 离散类数据挖掘的基准属性选择技术, IEEE 知识与数据工程学报 15(6) (2003) 1437–1447。
- [43] D. Hawkins, 过度拟合问题, 化学信息与计算机科学杂志44 (1) (2004) 1–12。
- [44] K. Healy, 《数据可视化：实用入门》，普林斯顿大学出版社, 2018 年。
- [45] H. Herodotou、H. Lim、G. Luo、N. Borisov、L. Dong、FB Cetin 和 S. Babu, 《Starfish：一种用于大数据分析的自调整系统》，载于：Cidr, 第 11 卷, 261–272, 2011 年。
- [46] D. Hoaglin、F. Mosteller 和 J. Tukey, 《理解稳健和探索性数据分析》，John Wiley and Sons, 1983 年。
- [47] R. Holte, 非常简单的分类规则在最常用的数据集上表现良好, 机器学习11 (1) (1993) 63–90, ISSN 1573-0565。
- [48] Q. Huang, S. Ritchie, M. Brozynska, M. Inouye, eQTL研究中的效力、错误发现率和赢家诅咒, bioRxiv (2017) 209171。
- [49] Q. Huang, X. Xia, D. Lo, 监督模型与无监督模型：全面审视基于努力的即时缺陷预测, 2017 年 IEEE 国际软件维护与演化会议 (ICSME), IEEE, 159–170, 2017 年。
- [50] J. Ioannidis, 为什么大多数已发表的研究结果都是错误的, PLoS Medicine 2 (8) (2005) e124–6。
- [51] M. Ivarsson, T. Gorschek, 一种评估技术评估的严谨性和工业相关性的方法, 实证软件工程 16(3)(2011) 365–395。

- [52] P. Johnson, M. Ekstedt, I. Jacobson, 软件工程理论在哪里? , IEEE 软件 29 (5) (2012) 96–96。
- [53] M. Jørgensen, T. Dybå, K. Liestøl, D. Sjøberg, 软件工程实验中的错误结果: 如何改进研究实践, J. of Syst. & 软件. 116 (2016) 133-145。
- [54] V. Kampenes, T. Dybå, J. Hannay, D. Sjøberg, “软件工程实验效应大小的系统评价”, 信息与软件技术 49 (11-12) (2007) 1073–1086。
- [55] B. Kitchenham, D. Budgen, P. Brereton, 《基于证据的软件工程和系统评价》, CRC Press, 佛罗里达州博卡拉顿, 美国, 2015 年。
- [56] B. Kitchenham, T. Dybå, M. Jørgensen, 《基于证据的软件工程》, 载于: 第 26 届 IEEE 国际软件工程会议 (ICSE 2004) , IEEE 计算机协会, 273–281, 2004 年。
- [57] B. Kitchenham, L. Madeyski, D. Budgen, J. Keung, P. Brereton, S. Charters, S. Gibbs, A. Pohthong, 经验软件工程的稳健统计方法, 经验软件工程 22 (2) (2017) 579–630。
- [58] B. Kitchenham, S. Pfleeger, L. Pickard, P. Jones, D. Hoaglin, K. El Emam, J. Rosenberg, 软件工程实证研究初步指南, IEEE 软件工程学报 28 (8) (2002) 721–734。
- [59] BA Kitchenham, OP Brereton, S. Owen, J. Butcher, C. Jefferies, 《结构化软件工程摘要的长度和可读性》, IET Software 2 (1) (2008) 37–45。
- [60] AJ Ko, TD Latoza, MM Burnett, 《软件工程工具与人类参与者对照实验实用指南》, 《经验软件工程》 20 (1) (2015) 110–141。
- [61] R. Krishna, A. Agrawal, A. Rahman, A. Sobran, T. Menzies, 《问题、缺陷和功能增强之间有何联系? (从 800 多个软件项目中汲取的经验教训) 》, CoRR abs/1710.08736, URL<http://arxiv.org/abs/1710.08736>。
- [62] R. Krishna, T. Menzies, 更简单的迁移学习 (使用 “领头羊”) , arXiv abs/1703.06218, 网址<http://arxiv.org/abs/1703.06218>。
- [63] R. Krishna, Z. Yu, A. Agrawal, M. Dominguez 和 D. Wolf, “BigSE” 项目: 从验证工业文本挖掘中获得的经验教训, 2016 IEEE/ACM 第二届大数据软件工程国际研讨会 (BIGDSE) (2016) 65-71。
- [64] D. Lakens, 等价检验: t 检验、相关性和元分析实用入门, 社会心理与人格科学 8 (4) (2017) 355– 362。

- [65] D. Lakens, E. Evers, 从混乱的海洋驶入稳定的走廊：提高研究信息价值的实用建议, 心理科学视角 9 (3) (2014) 278–292。
- [66] Y. LeCun, Y. Bengio, G. Hinton, 《深度学习》, 《自然》 521(7553) (2015) 436。
- [67] S. Lessmann, B. Baesens, C. Mues, S. Pietsch, 《软件缺陷预测的基准分类模型：一个建议的框架和新的发现》, IEEE 软件工程学报 34 (4) (2008) 485–496。
- [68] G. Liebchen, M. Shepperd, 《软件工程中的数据集和数据质量》, 载于：PROMISE 2008, ACM Press, 2008 年。
- [69] G. Liebchen, M. Shepperd, 《软件工程中的数据集和数据质量：八年后再》, 载于：第 12 届软件工程预测模型和数据分析国际会议论文集, ACM, 2016 年。
- [70] D. Lo, N. Nagappan, T. Zimmermann, 从业者如何看待软件工程研究的相关性, 载于：2015 年第 10 届软件工程基础联合会议论文集, ACM, 415–425, 2015 年。
- [71] E. Loken, A. Gelman, 测量误差与复制危机, Science 355 (6325) (2017) 584–585。
- [72] L. Madeyski, B. Kitchenham, “更广泛地采用可重复研究是否会有利于实证软件工程研究?”, 智能与模糊系统杂志 32 (2017) 1509–1521。
- [73] B. Manly, 《生物学中的随机化、引导程序和蒙特卡罗方法》, Chapman & Hall, 伦敦, 1997 年。
- [74] S. Maxwell, K. Kelley, J. Rausch, 参数估计中统计功效和准确度的样本大小规划, 《心理学年鉴》 59 (2008) 537–563。
- [75] G. McClelland, 在不增加样本量的情况下提高统计功效, 美国心理学家 55 (8) (2000) 963–964。
- [76] T. Menzies, J. Greenwald, A. Frank, 数据挖掘静态代码属性以学习缺陷预测器, IEEE 软件工程学报 33 (1) (2007) 2–13。
- [77] T. Menzies, M. Shepperd, 编辑部：软件工程预测可重复结果特刊, 2012 年。
- [78] N. Mittas, L. Angelis, 通过多重比较算法对软件成本估算模型进行排名和聚类, IEEE 软件工程学报 39 (4) (2013) 537–551。

- [79] M. Munafò, B. Nosek, D. Bishop, K. Button, C. Chambers, N. du Sert, U. Simonsohn, E. Wagenmakers, J. Ware, J. Ioannidis, 《可重复科学宣言》, 《自然-人类行为》 1 (2017) 0021。
- [80] G. Myatt、W. Johnson, 《理解数据 II: 数据可视化、高级数据挖掘方法和应用实用指南》, John Wiley & Sons, 2009 年。
- [81] V. Nair, A. Agrawal, J. Chen, W. Fu, G. Mathew, T. Menzies, L. Minku, M. Wagner, Z. Yu, 基于数据驱动搜索的软件工程, 载于: MSR'18, 2018。
- [82] R. Nickerson, 确认偏差: 一种以多种形式普遍存在的现象, 《普通心理学评论》 2 (2) (1998) 175-220。
- [83] 开放科学合作组织, 评估心理科学的可重复性, Science 349 (6251) (2015) aac4716-3。
- [84] K. Petersen, S. Vakkalanka, L. Kuzniarz, 软件工程中开展系统映射研究的指南: 更新, 信息与软件技术 64 (2015) 1-18。
- [85] M. Rosli, E. Tempero, A. Luxton-Reilly, “我们可以相信我们的结果吗? 一项关于数据质量的映射研究”, 载于: 第 20 届 IEEE 亚太软件工程会议 (APSEC'13) , 116-123, 2013 年。
- [86] P. Runeson, M. Höst, 软件工程案例研究的开展和报告指南, 实证软件工程 2 (14) (2009) 131-164。
- [87] A. Saltelli, S. Tarantola, F. Campolongo, 《敏感性分析作为建模的要素》, 《统计科学》 15(4) (2000) 377-395。
- [88] A. Sarkar, J. Guo, N. Siegmund, S. Apel, K. Czarnecki, 可配置系统性能预测的成本高效采样 (t) , 载于: 自动化软件工程 (ASE) , 2015 年第 30 届 IEEE/ACM 国际会议, IEEE, 342-352, 2015 年。
- [89] AS Sayyad、H. Ammar, 《基于帕累托最优搜索的软件工程 (POSBSE) : 文献综述》, 载于: 第二届实现软件工程人工智能协同效应的国际研讨会 (RAISE) , IEEE, 2013 年 21-27 日。
- [90] F. Schmidt、J. Hunter, 《荟萃分析方法: 纠正研究结果中的错误和偏见》, Sage Publications, 第 3 版, 2014 年。
- [91] W. Shadish、T. Cook、D. Campbell, 《广义因果推理的实验和准实验设计》, 霍顿·米夫林, 波士顿, 2002 年。
- [92] M. Shaw, 撰写优秀的软件工程研究论文, 载于: 第 25 届 IEEE 软件工程国际会议, IEEE 计算机协会, 726-736, 2003 年。

- [93] M. Shepperd, D. Bowes, T. Hall, 研究者偏见：机器学习在软件缺陷预测中的应用, IEEE 软件工程学报 40(6) (2014) 603-616。
- [94] M. Shepperd, Q. Song, Z. Sun, C. Mair, 数据质量:对 NASA 软件缺陷数据集的一些评论, IEEE 软件工程学报 39 (9) (2013) 1208-1215。
- [95] R. Silberzahn、E. Uhlmann、D. Martin、P. Anselmi、F. Aust、E. Awtrey、Š. Bahník、F. Bai、C. Bannard、E. Bonnier 等人,《众多分析师, 一个数据集: 明确分析选择的变化如何影响结果》URL<https://osf.io/j5v8f>。
- [96] J. Simmons, L. Nelson, U. Simonsohn, 假阳性心理学: 数据收集和分析中未公开的灵活性允许将任何事物呈现为重要事物, 心理科学 22 (11) (2011) 1359-1366。
- [97] D. Sjøberg、T. Dybå、B. Anda 和 J. Hannay,《软件工程理论构建》,《高级经验软件工程指南》, Springer, 312-336, 2008 年。
- [98] P. Smaldino, R. McElreath,《坏科学的自然选择》, 皇家学会开放科学 3 (9) (2016) 160384。
- [99] J. Snoek, H. Larochelle, R. Adams,《机器学习算法的实用贝叶斯优化》, 载于: 神经信息处理系统进展 (NIPS 2012, 2951-2959, 2012 年。
- [100] J. Spence、D. Stanley, 预测区间: 当你期待……时会发生什么, PLoS ONE 11(9) (2016) e0162874。
- [101] K.-J. Stol、B. Fitzgerald,《揭示软件工程理论》, 载于: 第二届 SEMAT 软件工程一般理论研讨会 (GTSE), IEEE, 5-14, 2013 年。
- [102] K.-J. Stol、P. Ralph、B. Fitzgerald,《软件工程研究中的扎根理论: 评论与指导方针》,《软件工程 (ICSE) 》, 2016 IEEE/ACM 第 38 届国际会议, IEEE, 120-131, 2016 年。
- [103] R. Storn, K. Price, 差分进化——一种简单有效的连续空间全局优化启发式方法, 全球优化杂志 11(4) (1997) 341-359, ISSN 1573-2916。
- [104] D. Szucs, J. Ioannidis, 对近期认知神经科学和心理学文献中已发表的效应大小和功效的实证评估, PLoS Biology 15 (3) (2017) e2000797。
- [105] C. Tantithamthavorn、S. McIntosh、A. Hassan、K. Matsumoto,《缺陷预测模型分类技术的自动参数优化》, 载于: IEEE/ACM 第 38 届国际软件工程会议 (ICSE 2016), IEEE, 321-332, 2016 年。

- [106] C. Tantithamthavorn, S. McIntosh, AE Hassan, K. Matsumoto, 《自动参数优化对缺陷预测模型的影响》, 《IEEE 软件工程学报》。
- [107] C. Theisen、M. Dunaiski、L. Williams 和 W. Visser, 《撰写优秀的软件工程研究论文: 重访》, 载于: IEEE/ACM 第 39 届国际软件工程会议指南 (ICSE-C), IEEE, 402–402, 2017 年。
- [108] D. Van Aken, A. Pavlo, G. Gordon, B. Zhang, 通过大规模机器学习实现自动数据库管理系统调优, 载于: 2017 年 ACM 国际数据管理会议论文集, ACM, 1009–1024, 2017 年。
- [109] P. Whigham, C. Owen, S. Macdonell, 《软件工作量估算的基线模型》, ACM 软件工程与方法论学报 24(3)。
- [110] R. Wilcox, 《稳健估计与假设检验导论》, Academic Press, 第 3 版, 2012 年。
- [111] C. Wohlin、P. Runeson、M. Höst、M. Ohlsson、B. Regnell、A. Wesslén, 软件工程实验, Springer Science & Business Media, 第 2 版, 2012 年。
- [112] Q. Zhang, H. Li, MOEA/D: 一种基于分解的多目标进化算法, IEEE Transactions on Evolutionary Computation 11(6) (2007) 712–731。
- [113] D. Zimmerman, “同时违反两个假设导致参数和非参数统计检验无效”, 《实验教育杂志》67 (1) (1998) 55–68。
- [114] T. Zimmermann、P. Weisgerber、S. Diehl 和 A. Zeller, 《挖掘版本历史以指导软件变更》, 载于: 第 26 届国际软件工程会议论文集, ICSE '04, IEEE 计算机协会, 美国华盛顿特区, ISBN 0-7695-2163-0, 563–572, URL<http://dl.acm.org/citation.cfm?id=998675.999460>, 2004 年。